

Java Basics - Object-Oriented Programming (OOP) Exam

Total Questions: 50 **Suggested Time:** 90 minutes **Instructions:**
Answer all questions. For multiple-choice questions, select the single best answer.

Part A - Multiple Choice (30 questions, 1 mark each)

1. Which of the following best defines a **class** in Java?
 - a. A running instance of an object
 - ☒ b. A blueprint/template for creating objects
 - c. A method that returns void
 - d. A built-in Java keyword
2. What is an **object** in Java?
 - a. A syntax error
 - ☒ b. An instance of a class
 - c. A type of loop
 - d. A reserved keyword
3. Which keyword is used to create an object in Java?
 - a. class
 - ☒ b. new
 - c. this
 - d. instance
4. Which of these is NOT one of the four main pillars of OOP?
 - a. Encapsulation
 - b. Inheritance
 - ☒ c. Compilation
 - d. Polymorphism
5. What is **encapsulation**?
 - ☒ a. Wrapping data and methods together and restricting direct access to data
 - b. Creating multiple classes with the same name
 - c. Converting one data type to another
 - d. Running two threads at once
6. Which access modifier restricts a member to be accessible only within its own class?
 - a. public
 - b. protected
 - ☒ c. private
 - d. default
7. Which access modifier allows access from any other class in any package?
 - a. private
 - b. protected
 - ☒ c. public
 - d. default (no modifier)
8. A constructor in Java:
 - a. Must always return a value
 - ☒ b. Has the same name as the class and no return type
 - c. Can only be called once per program
 - d. Must be declared static

9. What is a **default constructor**?
 - a. A constructor with parameters
 - ☒ b. A no-argument constructor automatically provided by Java if no constructor is defined
 - c. A constructor that only exists in abstract classes
 - d. A constructor that must be private
10. Which keyword is used to refer to the current object inside a method or constructor?
 - a. self
 - ☒ b. this
 - c. current
 - d. obj
11. What does **inheritance** allow a class to do?
 - a. Store multiple data types in one variable
 - ☒ b. Acquire fields and methods from another class
 - c. Run without a main method
 - d. Avoid using constructors
12. Which keyword is used to inherit a class in Java?
 - a. implements
 - ☒ b. extends
 - c. inherits
 - d. super
13. In Java, a class can extend how many other classes directly?
 - a. Zero
 - ☒ b. One
 - c. Two
 - d. As many as needed
14. What is the purpose of the **super** keyword?
 - a. To create a new object
 - ☒ b. To refer to the parent class's members or constructor
 - c. To declare a class as final
 - d. To stop inheritance
15. What is **method overriding**?
 - a. Defining multiple methods with the same name but different parameters in the same class
 - ☒ b. Redefining a parent class method in a child class with the same signature
 - c. Declaring a method as private
 - d. Calling a method twice
16. What is **method overloading**?
 - ☒ a. Having multiple methods in the same class with the same name but different parameter lists
 - b. Overriding a method from a parent class
 - c. Declaring a method final
 - d. Using recursion
17. Which of the following can be overloaded in Java?
 - a. Only constructors
 - b. Only regular methods
 - ☒ c. Both methods and constructors
 - d. Neither
18. What is **polymorphism**?
 - ☒ a. The ability of an object to take many forms
 - b. The process of hiding data
 - c. A type of loop structure
 - d. A way to declare arrays
19. Which type of polymorphism is achieved through method overriding?
 - a. Compile-time (static) polymorphism
 - ☒ b. Run-time (dynamic) polymorphism
 - c. Constructor polymorphism

- d. None of the above
- 20. What is an **abstract class**?
 - a. A class that cannot have any methods
 - ☒ b. A class that cannot be instantiated directly and may contain abstract methods
 - c. A class with only static methods
 - d. A class that must implement an interface
- 21. Which keyword declares an abstract method?
 - a. final
 - b. static
 - ☒ c. abstract
 - d. private
- 22. What is an **interface** in Java?
 - a. A class with only constructors
 - ☒ b. A contract that specifies methods a class must implement
 - c. A type of loop
 - d. A private helper class
- 23. Which keyword does a class use to implement an interface?
 - a. extends
 - ☒ b. implements
 - c. inherits
 - d. uses
- 24. Can a Java interface (pre-Java 8 style) contain method bodies?
 - a. Yes, always
 - ☒ b. No, traditionally interface methods are abstract by default
 - c. Only in the main method
 - d. Only for private methods
- 25. What is the main difference between an abstract class and an interface?
 - a. There is no difference
 - ☒ b. An abstract class can have constructors and instance variables with state; an interface traditionally cannot
 - c. Interfaces can be instantiated directly
 - d. Abstract classes cannot have any methods
- 26. What does the **static** keyword mean when applied to a variable?
 - ☒ a. The variable belongs to the class, not to individual objects
 - b. The variable can never change
 - c. The variable is private by default
 - d. The variable must be initialized in a constructor
- 27. What does the **final** keyword do when applied to a variable?
 - a. Makes the variable static
 - ☒ b. Prevents the variable's value from being changed after initialization
 - c. Makes the variable public
 - d. Deletes the variable after use
- 28. What happens if a class is declared **final**?
 - ☒ a. It cannot be extended (no subclasses allowed)
 - b. It cannot have any methods
 - c. It must be abstract
 - d. It becomes an interface
- 29. Which of these correctly describes **constructor chaining** using `this()`?
 - ☒ a. Calling another constructor of the same class from within a constructor
 - b. Calling a method from another class
 - c. Declaring two constructors with the same signature
 - d. Making a constructor static
- 30. What is the output of calling a method on an object reference that is `null`?
 - a. It returns 0

- b. It throws a NullPointerException
 - c. It silently does nothing
 - d. It automatically creates a new object
-

Part B - True or False (10 questions, 1 mark each)

- 31. A static method in a subclass can override a static method in its parent class the same way instance methods are overridden. (True/False) **F**
 - 32. A class can implement multiple interfaces at the same time. (True/False) **T**
 - 33. Constructors can be overloaded just like regular methods. (True/False) **T**
 - 34. Static methods can directly access non-static (instance) variables without an object reference. (True/False) **F**
 - 35. An abstract class can have a constructor even though it cannot be instantiated directly. (True/False) **T**
 - 36. Overriding requires the method name, return type (or covariant type), and parameter list to match the parent method. (True/False) **T**
 - 37. In Java, all classes implicitly inherit from the Object class unless they already extend another class. (True/False) **T**
 - 38. Encapsulation is typically achieved using private fields with public getter and setter methods. (True/False) **T**
 - 39. Java supports multiple inheritance of classes (a class extending two classes at once). (True/False) **F**
 - 40. A final method cannot be overridden by a subclass. (True/False) **T**
-

Part C - Short Answer / Code Output (10 questions, 2 marks each)

41. Write the output of the following code:

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}
```

dog barks

```

    }
}

```

42. Explain in 2-3 sentences the difference between **overloading** and **overriding**.

Method overloading means creating multiple methods with the same name in the same class but with different parameter lists.

43. Write the output of the following code:

Method overriding means redefining a method in a subclass using the same method signature as the parent class.

```

class Counter {
    static int count = 0;
    Counter() {
        count++;
    }
}

public class Main {
    public static void main(String[] args) {
        new Counter();
        new Counter();
        new Counter();
        System.out.println(Counter.count);
    }
}

```

44. Given the class below, write ONE line of code to create an object of Car and call its drive() method:

```

class Car {
    void drive() {
        System.out.println("Car is driving");
    }
}

```

*Car c = new Car();
c.drive();*

A compile-time error will occur because Shape is an abstract class.

45. What will happen if you try to instantiate the abstract class below directly with new Shape()? Explain why.

```

abstract class Shape {
    abstract double area();
}

```

Compile time ERROR

Abstract classes cannot be instantiated directly using the new keyword. A subclass must extend the abstract class and implement its abstract methods.

46. Write the output of the following code:

```

class Parent {
    Parent() {
        System.out.println("Parent constructor");
    }
}

class Child extends Parent {
    Child() {
        System.out.println("Child constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
    }
}

```

*Parent constructor
Child constructor*

47. Identify and correct the error in the following code:

```

public class Main {
    private int x = 5;
    public static void main(String[] args) {

```

main obj = new Main();
 System.out.println(~~(obj.x)~~);

48. Briefly explain why encapsulation is considered good practice in OOP (2-3 sentences).

Encapsulation protects data from direct access by making fields private and providing public getter and setter methods.

49. Write the output of the following code:

```
interface Greeting {
    void sayHello();
}
class English implements Greeting {
    public void sayHello() {
        System.out.println("Hello!");
    }
}
public class Main {
    public static void main(String[] args) {
        Greeting g = new English();
        g.sayHello();
    }
}
```

It improves security, makes the code easier to maintain, and helps prevent unintended changes to the object's data.

HELLO

50. Write the output of the following code:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
}
public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(2, 3));
        System.out.println(c.add(2.5, 3.5));
    }
}
```

5

End of Exam

6.0